

University of Engineering and Technology, Lahore



MR_Lab_Report_6
Submitted by: 2022_MC_27

Reactive Navigation Using Turtlebot3 LiDAR in Gazebo Environment

1. Lab Objective

- Obstacle detection using raw LiDAR scan data
- Stop-on-obstacle safety behavior
- Obstacle avoidance with intelligent turn-direction selection
- Wall following using proportional control (P-controller)
- Behavior sequencing combining all the above into a unified navigation system

2. Lab Introduction

2.1 Background

Reactive navigation is one of the fundamental paradigms in mobile robotics. Unlike deliberative navigation, which relies on pre-built maps and global path planning algorithms, reactive navigation generates motion commands directly and instantly from raw sensor inputs. This approach is inspired by biological systems where animals respond to stimuli without requiring conscious planning — for example, a hand withdrawing from a hot surface before the brain has fully processed the signal.

In this lab, the TurtleBot3 Burger robot is equipped with a 2D LiDAR sensor (LDS-01) that continuously scans the surrounding environment and publishes distance measurements on the `/scan` ROS 2 topic. These measurements are processed in real time to detect obstacles, estimate wall distances, and generate appropriate velocity commands published on the `/cmd_vel` topic. The robot has no map of its environment and makes all decisions based solely on current sensor readings.

2.2 ROS 2 Communication Model

The lab relies on the ROS 2 publish-subscribe communication model. Two core topics are used throughout all tasks:

- `/scan` (`sensor_msgs/msg/LaserScan`) — Published by the LiDAR driver. Contains the full 360-degree array of distance measurements. The node subscribes to this topic to perceive the environment.
- `/cmd_vel` (`geometry_msgs/msg/Twist`) — Subscribed to by the robot's motor controller. The node publishes linear and angular velocity commands to this topic to move the robot.

The callback-driven programming model in ROS 2 means that every time a new `LaserScan` message arrives, the `scan_callback()` function is automatically triggered, sensor data is processed, a decision is made, and a `Twist` command is published — all within a single callback execution.

2.4 LaserScan Data Structure

The `LaserScan` message contains a `ranges[]` array where each element represents the measured distance to the nearest surface at a specific angle. For the TurtleBot3 Burger LiDAR, index 0 corresponds to the forward direction (0 degrees), and indices increase counter-clockwise. This means:

- Index 0 and 359 — Directly in front of the robot
- Index 90 — Directly to the robot's left
- Index 180 — Directly behind the robot
- Index 270 — Directly to the robot's right

Sensor readings can be invalid (inf or NaN) when a surface is out of range or a reflection is missed. These must be replaced with safe default values before any distance comparisons are performed. In this lab, inf values are replaced with 10.0 m (interpreted as a clear path) and NaN values are replaced with 0.0 m.

4. Pre-Lab Setup

4.1 Launching the Simulation

The TurtleBot3 Burger model was launched in the default Gazebo world using the following terminal commands:

```
# Set robot model environment variable
export TURTLEBOT3_MODEL=burger

# Launch TurtleBot3 in Gazebo
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

The TurtleBot3 World environment contains walls and obstacles, making it ideal for testing reactive navigation behaviors such as obstacle avoidance and wall following.

4.2 Verifying the LiDAR Topic

The following commands were used to verify that the /scan topic was publishing correctly:

```
# List all active topics
ros2 topic list

# Echo raw LiDAR scan messages
ros2 topic echo /scan

# Optional: Visualize in RViz2
ros2 run rviz2 rviz2
```

6. Lab Task Implementations

Task 1: Scan Processing

6.1.1 Approach

The scan callback receives the LaserScan message and processes the ranges array in three steps: data cleaning, region extraction, and minimum distance computation. All inf/NaN values are first sanitized, then the 360-degree scan is divided into three directional windows, and the minimum distance in each region is computed and logged.

6.1.2 Region Definition

Region	Index Slice	Explanation
Front	ranges[:30] + ranges[-30:]	60-degree cone directly ahead (+/- 30 degrees)
Left	ranges[60:120]	60-degree window on the left side
Right	ranges[-120:-60]	60-degree window on the right side

6.1.3 Source Code

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
import numpy as np

class ScanProcessor(Node):
    def __init__(self):
        super().__init__('scan_processor')
        self.subscription = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)

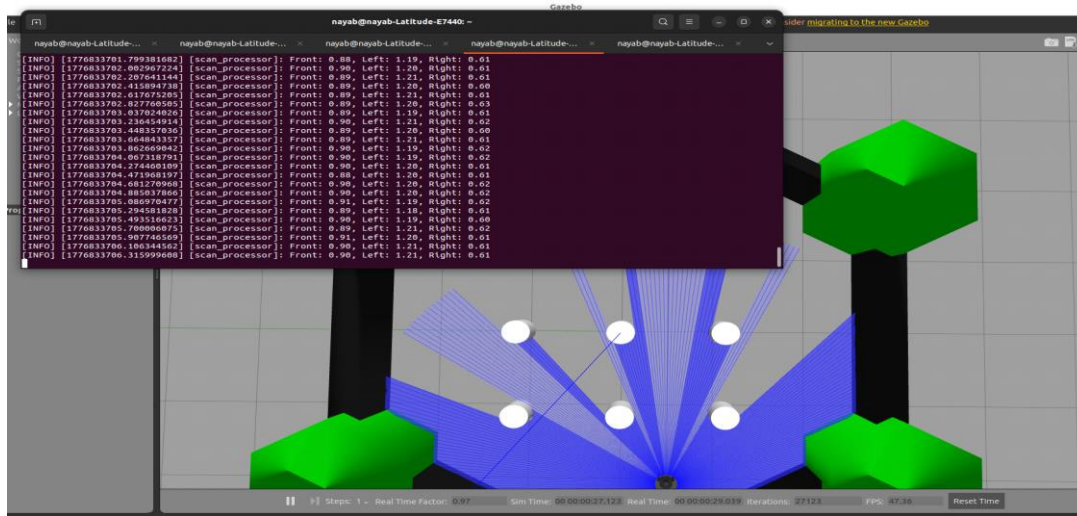
    def scan_callback(self, msg):
        ranges = np.array(msg.ranges)
        # Clean data: replace NaN with 0, inf with 10m
        ranges = np.nan_to_num(ranges, nan=0.0, posinf=10.0, neginf=0.0)
        # Define directional regions
        front = np.concatenate((ranges[:30], ranges[-30:]))
        left = ranges[60:120]
        right = ranges[-120:-60]
        # Compute minimum distances
        front_dist = np.min(front)
        left_dist = np.min(left)
        right_dist = np.min(right)
        self.get_logger().info(
            f'Front: {front_dist:.2f}, Left: {left_dist:.2f}, Right: {right_dist:.2f}')
        )
```

6.1.4 Expected Output

The node prints live distance readings to the terminal at every LiDAR scan cycle. Example output:

```
[INFO] [scan_processor]: Front: 1.24, Left: 0.87, Right: 2.10
[INFO] [scan_processor]: Front: 0.93, Left: 0.85, Right: 2.15
[INFO] [scan_processor]: Front: 0.62, Left: 0.84, Right: 2.20
```

6.1.5 Demonstration Screenshots



Task 2: Stop-on-Obstacle

6.2.1 Approach

A Twist publisher is added alongside the scan subscriber. A threshold of 0.5 m is defined. If front_dist exceeds the threshold, a forward velocity of 0.15 m/s is commanded. Otherwise, all velocity components are set to zero, causing the robot to stop. This forms the fundamental safety layer that all subsequent tasks build upon.

6.2.2 Decision Logic

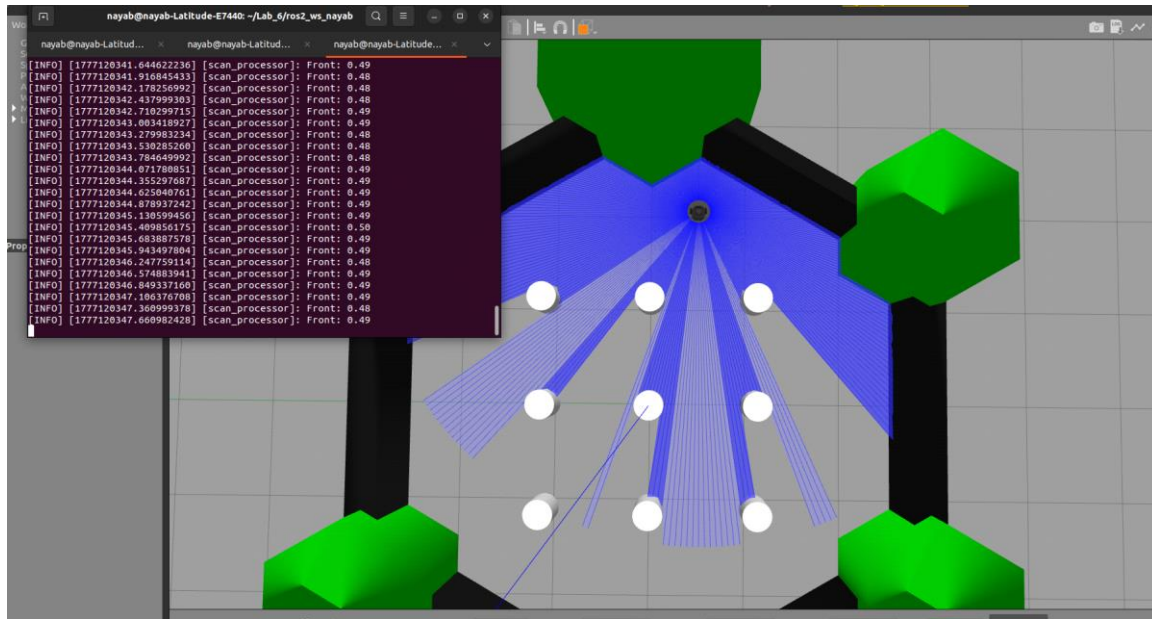
Condition	linear.x	angular.z	Behavior
front_dist > 0.5 m (clear)	0.15 m/s	0.0 rad/s	Move forward
front_dist <= 0.5 m (obstacle)	0.0 m/s	0.0 rad/s	Full stop

6.2.3 Source Code

```
self.threshold = 0.5    # stop distance in meters

def scan_callback(self, msg):
    ranges = np.array(msg.ranges)
    ranges = np.nan_to_num(ranges, nan=0.0, posinf=10.0, neginf=0.0)
    front = np.concatenate((ranges[:30], ranges[-30:]))
    front_dist = np.min(front)
    self.get_logger().info(f'Front: {front_dist:.2f}')
    twist = Twist()
    if front_dist > self.threshold:
        twist.linear.x = 0.15    # Move forward
        twist.angular.z = 0.0
    else:
        twist.linear.x = 0.0    # STOP
        twist.angular.z = 0.0
    self.publisher.publish(twist)
```

6.2.4 Demonstration Screenshots



Task 3: Obstacle Avoidance

6.3.1 Approach

Building on Task 2, when an obstacle is detected in the front region, the robot stops linear motion and rotates. The turn direction is chosen by comparing `left_dist` and `right_dist` — the robot turns toward the side with more free space. This ensures that the robot intelligently selects the optimal escape direction without requiring any map or prior knowledge.

6.3.2 Decision Logic

Condition	linear.x	angular.z	Behavior
front clear (> 0.6 m)	0.15 m/s	0.0 rad/s	Move forward
front blocked + left $>$ right	0.0 m/s	+0.6 rad/s	Turn left (CCW)
front blocked + right $\geq$ left	0.0 m/s	-0.6 rad/s	Turn right (CW)

6.3.3 Source Code

```
self.front_threshold = 0.6    # obstacle detection distance

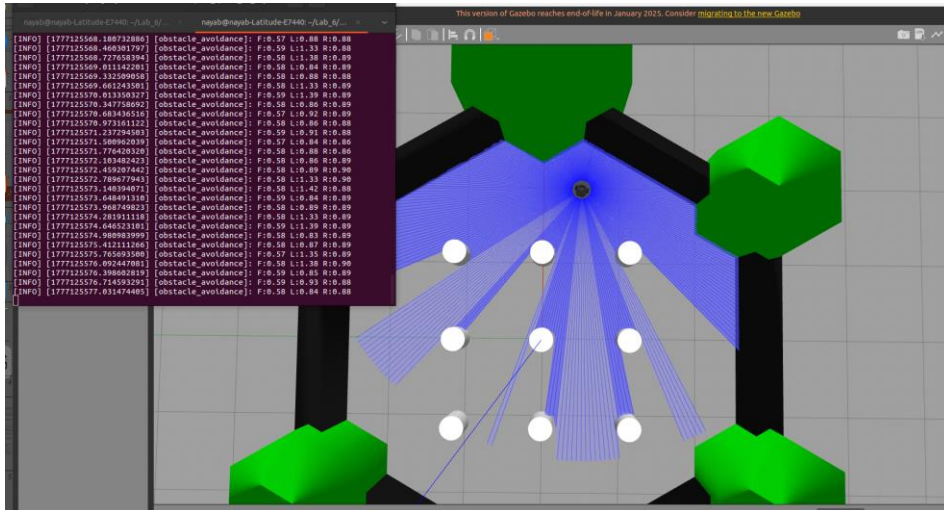
def scan_callback(self, msg):
    ranges = np.array(msg.ranges)
    ranges = np.nan_to_num(ranges, nan=10.0, posinf=10.0, neginf=0.0)
    front = np.concatenate((ranges[:30], ranges[-30:]))
    left = ranges[int(total*0.25):int(total*0.45)]
    right = ranges[int(total*0.55):int(total*0.75)]
    front_dist = np.min(front)
    left_dist = np.min(left)
    right_dist = np.min(right)
    twist = Twist()
    if front_dist < self.front_threshold:
        twist.linear.x = 0.0
        if left_dist > right_dist:    # More space on the left
            twist.angular.z = 0.6    # Turn left (CCW)
        else:
            twist.angular.z = -0.6    # Turn right (CW)
```

```

else:
    twist.linear.x = 0.15          # Move forward
    twist.angular.z = 0.0
    self.publisher.publish(twist)

```

6.3.4 Demonstration Screenshots



Task 4: Wall Following

6.4.1 Approach

A proportional (P) controller computes an angular correction based on the error between the desired wall distance and the measured right-side distance. A safety check triggers a hard left turn if the front distance becomes critically small. The controller ensures smooth path-following without abrupt direction changes.

6.4.2 P-Controller Equations

```

error      = desired_distance - right_dist
angular_z  = Kp * error              (Kp = 0.5)
angular_z  = clamp(angular_z, -0.4, +0.4)  rad/s

```

If $\text{right_dist} < \text{desired_distance}$: $\text{error} > 0 \Rightarrow \text{angular_z} > 0 \Rightarrow \text{turn left}$ (away from wall)

If $\text{right_dist} > \text{desired_distance}$: $\text{error} < 0 \Rightarrow \text{angular_z} < 0 \Rightarrow \text{turn right}$ (toward wall)

6.4.3 Parameters

Parameter	Value	Purpose
desired_distance	0.5 m	Target distance from right wall
Kp	0.5	Proportional gain (empirically tuned)
front_threshold	0.45 m	Emergency front safety distance
linear.x	0.15 m/s	Constant forward speed
angular clamp	[-0.4, 0.4] rad/s	Prevents oscillation / overshoot

6.4.4 Source Code

```

self.desired_distance = 0.5
self.Kp = 0.5
self.front_threshold = 0.45

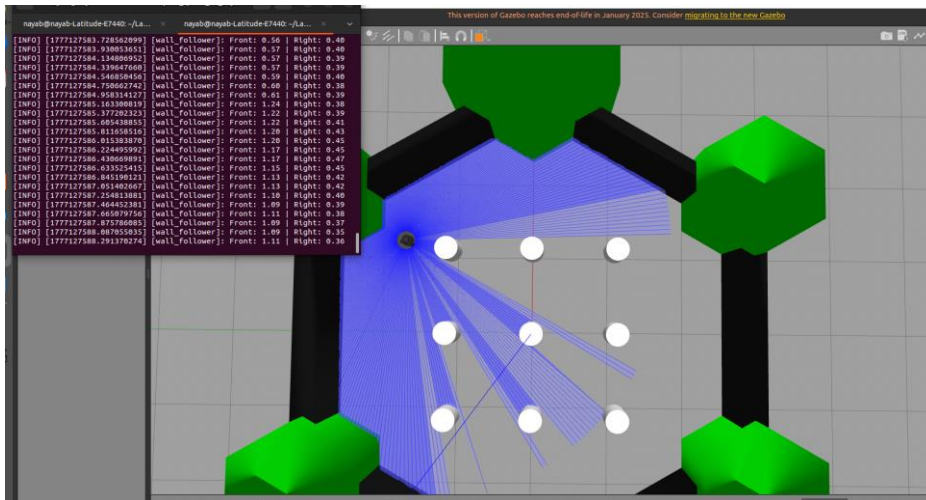
```

```

def scan_callback(self, msg):
    ranges = np.array(msg.ranges)
    ranges = np.nan_to_num(ranges, nan=10.0, posinf=10.0, neginf=0.0)
    front = np.concatenate((ranges[:25], ranges[-25:]))
    right = ranges[int(total * 0.65):int(total * 0.85)]
    front_dist = np.min(front)
    right_dist = np.min(right)
    twist = Twist()
    # Safety: hard left turn if front too close
    if front_dist < self.front_threshold:
        twist.linear.x = 0.0
        twist.angular.z = 0.6
        self.publisher.publish(twist)
        return
    # Proportional wall following
    error = self.desired_distance - right_dist
    twist.linear.x = 0.15
    angular = self.Kp * error
    angular = max(min(angular, 0.4), -0.4) # Clamp
    twist.angular.z = angular
    self.publisher.publish(twist)

```

6.4.5 Demonstration Screenshots



Task 5: Behavior Sequencing

6.5.1 Approach

A priority-ordered decision tree is implemented. The highest priority behavior (obstacle avoidance) is checked first. If not triggered, wall following is checked. If neither applies, the robot moves straight at a slightly higher speed. This design ensures safety is never compromised by lower-priority behaviors.

6.5.2 Behavior Priority Table

Priority	Behavior	Trigger Condition	Action
1	Obstacle Avoidance	front_dist < 0.5 m	Stop + turn toward clearer side
2	Wall Following (Right)	right_dist < 0.8 m	P-controller angular

			correction
3	Move Straight	No trigger met	linear.x = 0.2 m/s forward

6.5.3 Noise Filtering

A `get_filtered_min()` function removes zero/noise readings (< 0.05 m) before taking the minimum distance, preventing false obstacle detections from sensor artifacts or reflective surfaces.

```
def get_filtered_min(self, data):
    data = data[data > 0.05]    # Remove near-zero noise
    if len(data) == 0:
        return 10.0            # No valid readings = assume clear
    return np.min(data)
```

6.5.4 Full Decision Logic

```
def scan_callback(self, msg):
    # ... data cleaning and region extraction ...
    front_dist = self.get_filtered_min(front)
    left_dist  = self.get_filtered_min(left)
    right_dist = self.get_filtered_min(right)
    twist = Twist()

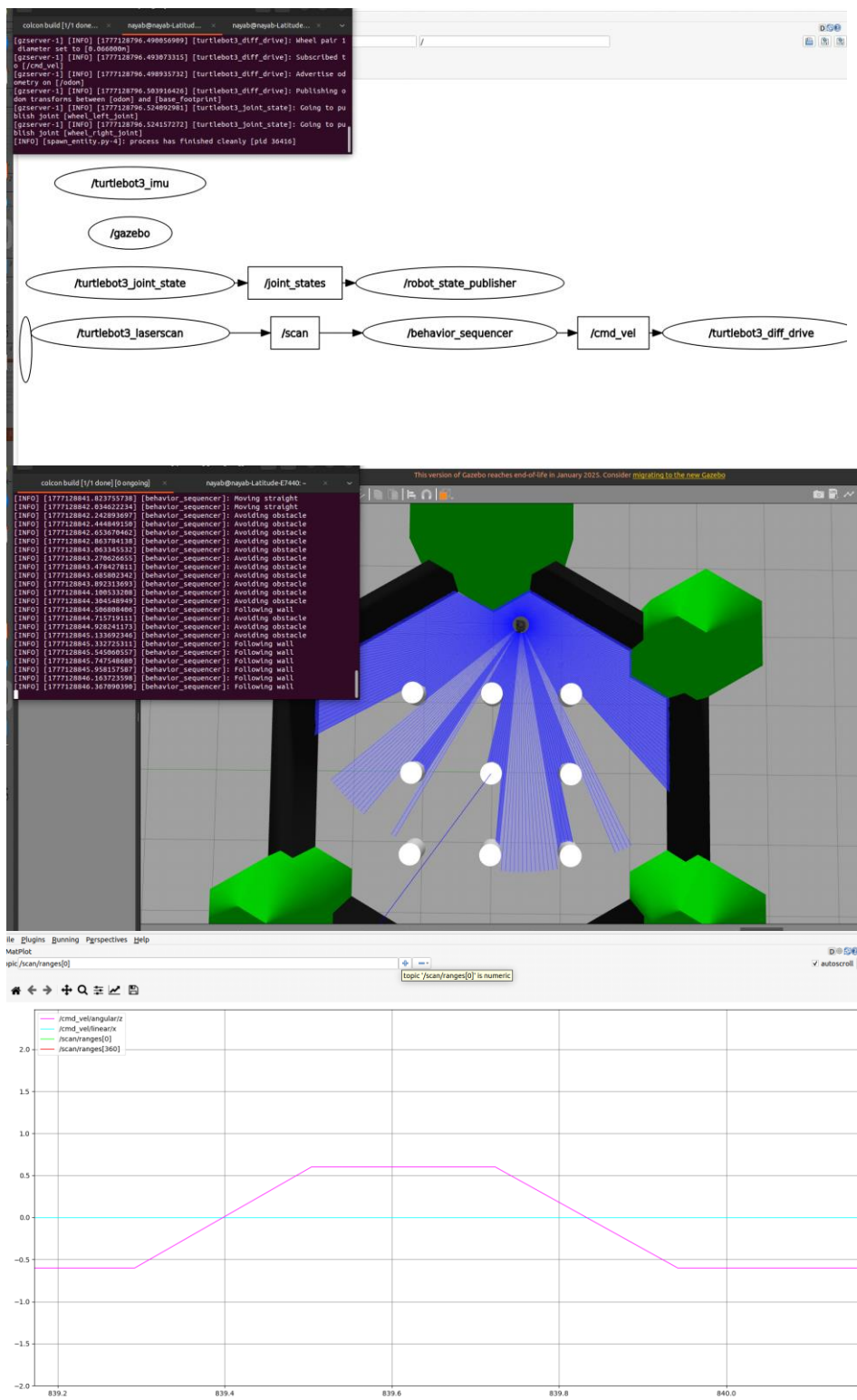
    # PRIORITY 1: Obstacle avoidance
    if front_dist < self.front_threshold:
        twist.linear.x = 0.0
        twist.angular.z = 0.6 if left_dist > right_dist else -0.6
        self.get_logger().info('Avoiding obstacle')

    # PRIORITY 2: Wall following (right wall detected)
    elif right_dist < self.wall_detect_dist:
        error = self.desired_wall_dist - right_dist
        twist.linear.x = 0.15
        angular = self.Kp * error
        twist.angular.z = max(min(angular, 0.5), -0.5)
        self.get_logger().info('Following wall')

    # PRIORITY 3: Default - move straight
    else:
        twist.linear.x = 0.2
        twist.angular.z = 0.0
        self.get_logger().info('Moving straight')

    self.publisher.publish(twist)
```

6.5.5 Demonstration Screenshots



7. Completed Boilerplate Node (lidar_navigator.py)

The lab manual provided a skeleton node with TODO markers. The completed implementation with all blanks filled is shown below:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import Twist
import numpy as np

class LidarNavigator(Node):
    def __init__(self):
        super().__init__('lidar_navigator')
        self.subscription = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.publisher = self.create_publisher(Twist, '/cmd_vel', 10)
        # Thresholds
        self.front_threshold = 0.5
        self.side_threshold = 0.5

    def scan_callback(self, msg):
        ranges = np.array(msg.ranges)
        # TODO 1: Clean data
        ranges = np.nan_to_num(ranges, nan=0.0, posinf=10.0, neginf=0.0)
        # TODO 2: Define regions
        front = np.concatenate((ranges[:30], ranges[-30:]))
        left = ranges[60:120]
        right = ranges[-120:-60]
        front_dist = np.min(front)
        left_dist = np.min(left)
        right_dist = np.min(right)
        twist = Twist()
        # TODO 3: Obstacle logic
        if front_dist < self.front_threshold:
            # TODO 4: Turn direction
            if left_dist > right_dist:
                twist.angular.z = 0.5 # Turn left
            else:
                twist.angular.z = -0.5 # Turn right
            twist.linear.x = 0.0
        else:
            # TODO 5: Forward motion
            twist.linear.x = 0.15
            twist.angular.z = 0.0
        self.publisher.publish(twist)

def main(args=None):
    rclpy.init(args=args)
    node = LidarNavigator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

8. ROS 2 Node Graph (rqt_graph)

The `rqt_graph` tool was used to visualize the ROS 2 computation graph. The command to launch it is:

```
ros2 run rqt_graph rqt_graph
```

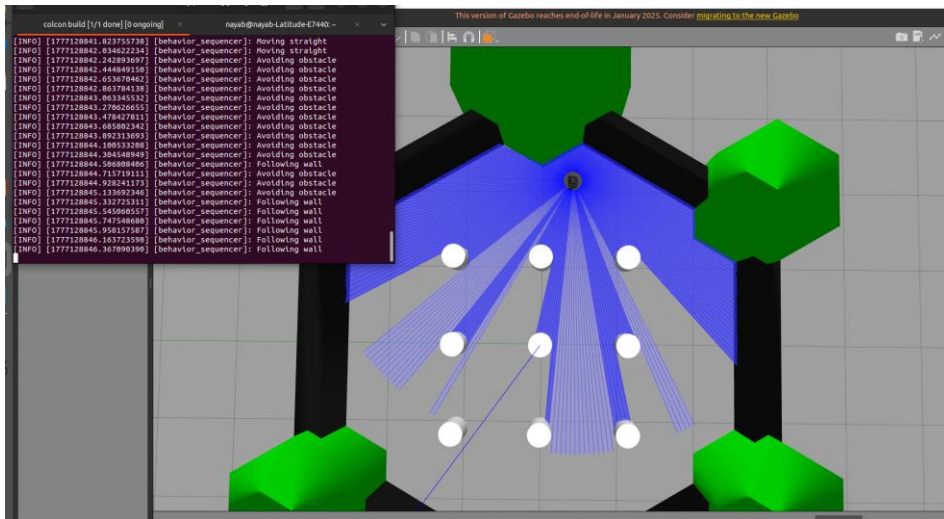
The expected graph shows the following node-topic connections:

Node	Subscribes To	Publishes To
scan_processor	/scan	(logging only — Task 1)
obstacle_avoidance	/scan	/cmd_vel
wall_follower	/scan	/cmd_vel
behavior_sequencer	/scan	/cmd_vel
/turtlebot3_diff_drive	/cmd_vel	(wheel odometry)

9. Observations & Analysis

9.1 Robot Behavior Near Obstacles

When the front_dist dropped below the threshold, the robot stopped linear motion immediately and began rotating. The turn direction was consistently correct — the robot reliably turned toward the side with more free space. The decision was made within a single scan cycle (~200 ms), resulting in a responsive and safe behavior. However, at very sharp corner angles, the robot occasionally required 2–3 seconds of rotation before the front region was cleared enough to resume forward motion.



9.2 Oscillations and Instability

Oscillations were observed primarily in the wall-following task (Task 4). When K_p was set too high (e.g., $K_p = 1.0$), the angular correction overshoot, causing the robot to swing back and forth while trying to maintain the desired distance. The solution involved three adjustments:

- Reducing K_p to 0.5 (empirically tuned through trial and error)
- Clamping angular.z to the range $[-0.4, 0.4]$ rad/s to prevent excessive corrections
- Maintaining a low, constant linear speed of 0.15 m/s to reduce instability from high-speed turns

After applying these tuning steps, the robot traveled smoothly along the wall with minimal oscillation and stable heading.

9.3 Effect of Changing Threshold Values

front_threshold	Too Low (0.2 m)	Too High (1.0 m)	Recommended
Effect on behavior	Robot gets dangerously close to walls before stopping	Robot turns too early; unable to enter corridors	0.5 m — consistent safe stopping

Kp (wall following)	Low (0.1)	High (1.0)	Recommended
Effect on behavior	Very sluggish correction; drifts away from wall over time	Heavy oscillation; unstable zig-zag path	0.5 — smooth, stable following

9.4 Sensor Noise Handling

Raw LiDAR readings occasionally contained zeros or values below 0.05 m due to sensor artifacts and highly reflective surfaces in the Gazebo environment. The `get_filtered_min()` function in Task 5 effectively eliminated these by filtering out values below 0.05 m before taking the minimum, dramatically reducing false positive obstacle detections and improving the stability of the behavior sequencer.

12. Conclusion

This lab provided comprehensive hands-on experience in implementing reactive navigation using LiDAR sensor data within the ROS 2 framework and Gazebo simulation environment. Starting from raw sensor data processing and building up through five progressive tasks, a complete reactive navigation system was successfully developed and demonstrated.

Key technical achievements:

1. Successfully processing LaserScan messages and extracting directional distance information through region segmentation (front, left, right)
2. Implementing a safe stop-on-obstacle behavior as the fundamental safety layer of the system
3. Developing an intelligent obstacle avoidance system that dynamically selects turn direction based on available clearance
4. Applying proportional control theory to achieve smooth wall following, with empirically tuned parameters that eliminate oscillation
5. Integrating all behaviors into a priority-based sequencer that transitions seamlessly between navigation modes

Challenges encountered and overcome:

- Handling NaN and inf values in LiDAR data required careful use of `np.nan_to_num()` to prevent incorrect distance calculations
- Tuning the proportional gain Kp for wall following required several iterations — too high caused oscillation, too low caused wall drift
- Selecting appropriate region boundaries (index slicing) was critical; incorrect slicing caused the robot to react to wrong directions
- The behavior sequencer required careful threshold ordering to prevent conflict between wall-following and obstacle-avoidance modes

The concepts and skills developed in this lab — sensor data processing, reactive decision-making, proportional control, and behavior arbitration — form a solid foundation for more advanced mobile robotics topics including SLAM (Simultaneous Localization and Mapping), global path planning with Nav2, and sensor fusion combining LiDAR with IMU and camera data.